Le jeu des allumettes — Corrigé

Conditions, boucles et interaction en Python — versions brute, guidée et ultra guidée

Comment lire ce corrigé

Pour chaque partie, le programme donné répond aux trois niveaux : c'est le squelette de la version guidée complété, et le résultat final de la version ultra guidée. Les réponses aux petites questions « À toi ! » sont données à part, et chaque partie se termine par des cas de test pour vérifier une copie.

Tous les programmes ont été exécutés. La partie graphique a été testée avec une bibliothèque nsi_ui simulée, qui reproduit label, button, entry, set_text et get_int : la logique du jeu est validée, mais pas l'affichage réel de la fenêtre.

Partie 1 — Version console

Programme complet (une partie)

```
allumettes = 20
tour_joueur = True

while allumettes > 0:
    print(f"Il reste {allumettes} allumettes")
    if tour_joueur:
        ok = False
        while not ok:
            try:
                prise = int(input("Combien en prends-tu (1 à 3) ? "))
                if 1 <= prise <= 3 and prise <= allumettes:
                    ok = True
                else:
                    if prise < 1 or prise > 3:
                        print("Tu dois choisir 1, 2 ou 3 !")
                    else:
                        print(f"Il ne reste que {allumettes} allumettes !")
            except ValueError:
                print("Ce n'est pas un nombre !")
    else:
        prise = (allumettes - 1) % 4
        if prise == 0:
            prise = 1
        print(f"L'ordinateur en prend {prise}")
    allumettes = allumettes - prise
    tour_joueur = not tour_joueur

if tour_joueur:
    print("L'ordinateur a pris la dernière : tu as gagné !")
else:
    print("Tu as pris la dernière : perdu !")
```

Le squelette de la version guidée se complète ainsi : while allumettes > 0 pour la boucle de jeu, if tour_joueur pour distinguer le joueur de l'ordinateur, while not ok pour la saisie, puis prise = (allumettes - 1) % 4 avec if prise == 0: prise = 1 pour l'ordinateur, allumettes = allumettes - prise et tour_joueur = not tour_joueur. La condition de validation est 1 <= prise <= 3 and prise <= allumettes ; la variante if / elif / else de la version ultra guidée (un message par cas) est tout aussi correcte.

Réponses aux « À toi ! » (version ultra guidée)

prise	prise < 1	prise > 3	prise < 1 or prise > 3	Résultat
0	True	False	True	Interdit
2	False	False	False	Autorisé
3	False	False	False	Autorisé
5	False	True	True	Interdit
-1	True	False	True	Interdit

Étape	Question	Réponse
3	Pourquoi or et pas and ?	Un nombre ne peut pas être à la fois < 1 et > 3 : avec and, la condition serait toujours fausse et tout serait autorisé. Il suffit d'une des deux parties vraie : or.
4	Que se passe-t-il si on oublie de redemander dans la boucle ?	prise ne change jamais : la condition reste vraie et la boucle tourne à l'infini (Ctrl+C pour l'arrêter).
6	À quoi sert le try / except ?	Si la saisie n'est pas un nombre, int() provoque une ValueError : Python saute dans le except, ok reste False et la boucle redemande.
7 / 9	Si tour_joueur vaut True après la boucle, qui a joué en dernier ?	L'ordinateur : tour_joueur a été inversé après le dernier coup. Il a pris la dernière allumette, donc il perd et le joueur gagne.
9	Où placer l'annonce du gagnant ?	Après la boucle de jeu, sans décalage, au même niveau que le while : le message s'affiche une seule fois.
10	Pourquoi allumettes = 20 dans la boucle « rejouer » ?	Tout ce qui doit être remis à zéro à chaque partie va dans la boucle ; sinon, à la 2e partie, il resterait 0 allumette.
10	Pourquoi and pour rejouer != 0 and rejouer != 1 ?	Avec or, la condition serait toujours vraie (1 != 0 est vrai) : la boucle ne s'arrêterait jamais.
11	Où créer victoires_joueur et victoires_ordi ?	Avant la grande boucle : dedans, elles seraient remises à 0 à chaque partie et le score ne se cumulerait pas.

Avec rejouer et score (étapes 10 et 11)

```
victoires_joueur = 0
victoires_ordi = 0
rejouer = 1

while rejouer == 1:
    allumettes = 20
    tour_joueur = True

    while allumettes > 0:
        print(f"Il reste {allumettes} allumettes")
        if tour_joueur:
            ok = False
            while not ok:
                try:
                    prise = int(input("Combien en prends-tu (1 à 3) ? "))
                    if prise < 1 or prise > 3:
                        print("Tu dois choisir 1, 2 ou 3 !")
                    elif prise > allumettes:
                        print(f"Il ne reste que {allumettes} allumettes !")
                    else:
                        ok = True
                except ValueError:
                    print("Ce n'est pas un nombre !")
        else:
            prise = (allumettes - 1) % 4
            if prise == 0:
                prise = 1
            print(f"L'ordinateur en prend {prise}")
        allumettes = allumettes - prise
        tour_joueur = not tour_joueur

    if tour_joueur:
        print("L'ordinateur a pris la dernière : tu as gagné !")
        victoires_joueur = victoires_joueur + 1
    else:
        print("Tu as pris la dernière : perdu !")
        victoires_ordi = victoires_ordi + 1
    print(f"Score : joueur {victoires_joueur} - ordinateur {victoires_ordi}")

    rejouer = int(input("Tape 1 pour rejouer, 0 pour arrêter : "))
    while rejouer != 0 and rejouer != 1:
        rejouer = int(input("Réponds 1 ou 0 : "))

print("Merci d'avoir joué !")
```

Question bonus : combien prendre au premier tour ?

Réponse : 3 allumettes (20 − 3 = 17). Les nombres de la forme 4k + 1 (1, 5, 9, 13, 17) sont perdants pour celui qui doit jouer : quel que soit son coup x (1, 2 ou 3), l'adversaire reprend 4 − x et retombe sur un nombre de la forme 4k + 1, jusqu'à lui laisser 1 allumette. L'ordinateur applique cette stratégie : (allumettes − 1) % 4 est exactement ce qu'il faut retirer pour retomber sur 4k + 1. Quand ce calcul donne 0, il est déjà sur un nombre perdant et prend 1 allumette en espérant une erreur du joueur.

Tour	Reste	Le joueur prend	Reste	L'ordinateur prend	Reste
1	20	3	17	1	16
2	16	3	13	1	12
3	12	3	9	1	8
4	8	3	5	1	4
5	4	3	1	1 (la dernière : il perd)	0

Si le joueur prend 1 ou 2 au premier tour, l'ordinateur lui laisse 17 : le joueur est alors dans une position perdante tant que l'ordinateur ne se trompe pas, et il n'en fait aucune.

Cas de test (partie 1)

Saisies	Résultat attendu
« trois », puis 7, puis 0	Un message « Ce n'est pas un nombre ! » puis deux messages « Tu dois choisir 1, 2 ou 3 ! » ; le jeu continue sans planter.
Quand il reste 2 allumettes : 3, puis 2	« Il ne reste que 2 allumettes ! », puis le coup 2 est accepté.
3, 3, 3, 3, 3	Le joueur gagne : « L'ordinateur a pris la dernière : tu as gagné ! ».
Six fois 1	Le joueur prend la dernière : « Tu as pris la dernière : perdu ! ».
Après une partie : 5, puis 0	« Réponds 1 ou 0 : », puis « Merci d'avoir joué ! ».

Partie 2 — Version graphique (nsi_ui)

Programme complet

```
from nsi_ui import *

allumettes = 20
fini = False
victoires_joueur = 0
victoires_ordi = 0

def jouer(prise):
    global allumettes, fini, victoires_joueur, victoires_ordi
    if fini:
        set_text(message, "Partie terminée !")
    elif prise > allumettes:
        set_text(message, f"Il ne reste que {allumettes} allumettes !")
    else:
        allumettes = allumettes - prise
        if allumettes == 0:
            set_text(message, "Tu as pris la dernière : perdu !")
            fini = True
            victoires_ordi = victoires_ordi + 1
        else:
            ordi = (allumettes - 1) % 4
            if ordi == 0:
                ordi = 1
            allumettes = allumettes - ordi
            if allumettes == 0:
                set_text(message, "Tu as gagné !")
                fini = True
                victoires_joueur = victoires_joueur + 1
            else:
                set_text(message, f"L'ordinateur en a pris {ordi}. À toi !")
    set_text(reste, f"Il reste {allumettes} allumettes")
    set_text(score, f"Score : joueur {victoires_joueur} - ordinateur
{victoires_ordi}")

def rejouer():
    global allumettes, fini
    allumettes = 20
    fini = False
    set_text(reste, f"Il reste {allumettes} allumettes")
    set_text(message, "Nouvelle partie : à toi de jouer !")

begin_vertical()
reste = label("Il reste 20 allumettes")
begin_horizontal()
button("Prendre 1", jouer, 1)
button("Prendre 2", jouer, 2)
button("Prendre 3", jouer, 3)
end_horizontal()
message = label("À toi de jouer !")
button("Rejouer", rejouer)
score = label("Score : joueur 0 - ordinateur 0")
end_vertical()
main_loop()
```

Sans le bonus (version brute et guidée), on retire victoires_joueur, victoires_ordi, le label score, le bouton Rejouer, la fonction rejouer et les deux lignes qui les utilisent dans jouer ; le reste est identique.

Réponses aux « À toi ! » et pièges

Point	Réponse
Ordre du programme	from nsi_ui import *, puis les variables, puis les fonctions de rappel, puis l'interface, et main_loop() en dernière ligne.
button("Prendre 1", jouer, 1)	On donne le nom de la fonction sans parenthèses : avec jouer(), Python l'appellerait une seule fois au lancement. Le 3e paramètre est transmis à jouer à chaque clic.
global allumettes, fini, ...	Obligatoire dès qu'une fonction modifie une variable créée en dehors. Sans cette ligne : UnboundLocalError au premier clic, et rien ne change dans la fenêtre. global accepte plusieurs variables séparées par des virgules.
begin_... / end_...	Chaque begin_vertical() ou begin_horizontal() doit avoir son end_... : les trois boutons sont dans un conteneur horizontal, lui-même dans un conteneur vertical.
Test allumettes == 0	Deux fois : juste après le coup du joueur, et juste après celui de l'ordinateur. Sinon, quand le joueur prend la dernière, l'ordinateur jouerait quand même et le nombre deviendrait négatif.
Coup interdit	Seul prendre plus que ce qu'il reste est possible avec des boutons : on affiche un message et on ne fait rien d'autre. Pas de boucle while : le programme attend le prochain clic (programmation réactive).
fini	Une fois la partie terminée, if fini: affiche « Partie terminée ! » et les boutons ne font plus rien jusqu'à « Rejouer ».
rejouer()	global allumettes, fini ; remet allumettes à 20 et fini à False ; puis set_text sur les deux labels (sans set_text, la fenêtre garderait l'ancien texte).
Étape 10 : remettre les victoires à 0 dans rejouer() ?	Non : le score doit se cumuler d'une partie à l'autre. Il faut en revanche ajouter victoires_joueur et victoires_ordi au global de jouer (pas de rejouer, qui n'y touche pas).

Cas de test (partie 2)

Clics	Résultat attendu
« Prendre 3 » au départ	« Il reste 16 allumettes » : 20 − 3 = 17, puis l'ordinateur en prend 1 (message « L'ordinateur en a pris 1. À toi ! »).
« Prendre 3 » quatre fois de plus	Il reste 0 allumette et le message « Tu as gagné ! » ; le score passe à 1 – 0.
Un clic de plus sur « Prendre 1 »	« Partie terminée ! » et rien ne change.
« Rejouer », puis « Prendre 1 » cinq fois	Il reste 1 allumette (l'ordinateur a pris 3 à son dernier tour).
Puis « Prendre 3 »	« Il ne reste que 1 allumettes ! » : le nombre ne change pas.
Puis « Prendre 1 »	« Tu as pris la dernière : perdu ! » ; le score passe à 1 – 1.

Partie 3 — Champ de saisie et try / except

On garde les variables et la fonction jouer de la partie 2 (version sans score : le label score n'existe pas ici). Seule l'interface change :

```
def clic_jouer():
    try:
        prise = get_int(champ)
        if prise < 1 or prise > 3:
            set_text(message, "Tu dois choisir 1, 2 ou 3 !")
        else:
            jouer(prise)
            set_text(champ, "")      # bonus : vide le champ après le coup
    except ValueError:
        set_text(message, "Ce n'est pas un nombre !")

def entree(texte):                   # bonus : la touche Entrée joue aussi
    clic_jouer()

begin_vertical()
reste = label("Il reste 20 allumettes")
champ = entry("Combien en prends-tu (1 à 3) ?", entree)
button("Jouer", clic_jouer)
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Point	Réponse
Où est l'erreur ?	get_int(champ) convertit le texte avec int() : si le joueur tape « trois », une ValueError est levée, comme avec int(input()) en console.
Que contient le try ?	La lecture du champ, le test entre 1 et 3 et l'appel de jouer(prise) : tout ce qui risque de lever l'erreur ou en dépend. Le except ValueError affiche « Ce n'est pas un nombre ! ».
Pourquoi plus de while ?	On affiche simplement l'erreur : le programme attend le prochain clic, le joueur corrige sa saisie et reclique. C'est la programmation réactive.
Pourquoi jouer ne change pas ?	Elle vérifie déjà s'il reste assez d'allumettes (prise > allumettes). Le test entre 1 et 3 est fait avant par clic_jouer.
Bouton « Jouer »	button("Jouer", clic_jouer) : le nom de la fonction, sans parenthèses et sans paramètre.

Cas de test (partie 3)

Saisie puis clic sur « Jouer »	Résultat attendu
« trois » ou champ vide	« Ce n'est pas un nombre ! » ; rien ne change.
5	« Tu dois choisir 1, 2 ou 3 ! » ; il reste toujours 20 allumettes.
3	« Il reste 16 allumettes » (l'ordinateur en prend 1) ; le champ est vidé.
3, puis touche Entrée	Même résultat qu'un clic sur « Jouer » : il reste 12 allumettes.